

Do Transformers Learn Base-Agnostic Arithmetic?

A Paragraph-by-Paragraph Study Guide — Original Text (highlighted) paired with Plain-English Explanations

1. Motivation

ORIGINAL TEXT

Small transformers trained on arithmetic tasks have been shown to develop clean, interpretable internal algorithms after a grokking phase transition [Power et al., 2022, Nanda et al., 2023]. A fundamental question that this body of work has not addressed is: does the learned algorithm operate on abstract numerical magnitude, or is it tied to the specific surface representation of numbers used during training?

IN PLAIN ENGLISH

Earlier researchers found that tiny AI models trained to do math eventually stop just memorizing and start using a real, understandable internal method — this delayed switch is called 'grokking'. But no one has asked: does the model actually understand the abstract idea of a number, or has it just learned a trick tied to the specific digit symbols (like '5' or '13') it was trained on?

ORIGINAL TEXT

The integer five can be written as 5 (decimal), 101 (binary), 5 (octal, coincidentally the same digit), or 5 (hexadecimal, also the same digit) — but for addition problems involving carries, the surface form determines the token sequence in a way that is highly non-trivial. A model trained on binary addition $1101 + 1011 = 11000$ and decimal addition $13 + 11 = 24$ is computing the same abstract operation, but the carry structure, the token sequence length, and the digit vocabulary are all different.

IN PLAIN ENGLISH

The number five looks almost the same across several number systems, but once you start adding bigger numbers that require 'carrying' digits, the way that carry plays out looks completely different depending on the base. Binary addition ($1101 + 1011$) and decimal addition ($13 + 11$) are mathematically the exact same kind of operation, but they use different symbols, different sequence lengths, and different carry patterns — so a model has to handle very different-looking inputs to do 'the same thing.'

ORIGINAL TEXT

Prior mechanistic interpretability work on arithmetic has studied a single numeral system at a time [Nanda et al., 2023, Kantamneni and Tegmark, 2025, Quirke and Barez, 2024]. It therefore cannot distinguish between a model that has learned an abstract positional-arithmetic algorithm and one that has learned a surface-specific algorithm that happens to generalize within a base. We address this gap directly. We train small transformers from scratch on addition in multiple numeral bases simultaneously, with a shared character-level tokenizer, and apply causal circuit analysis to determine whether the same internal components are responsible for addition across all bases.

IN PLAIN ENGLISH

Every previous 'open the AI's brain' study of arithmetic only ever tested one number system (always decimal). So those studies literally cannot tell the difference between 'the model learned real math' and 'the model learned a decimal-only shortcut that just looks like real math.' This paper closes that gap: train one model on four number systems at once, then directly test (using causal experiments, not just observation) whether the same internal components are doing the work in every base.

2. Significance

ORIGINAL TEXT

This work provides the first causally grounded account of whether arithmetic circuits in transformers are tied to surface representation or encode a genuinely base-agnostic algorithm. By training on multiple bases with a balanced corpus, we eliminate data imbalance as a confound. The four bases chosen — binary, octal, decimal, and hexadecimal — form a natural gradient of token overlap with each other, enabling a structured test of whether circuit sharing correlates with surface similarity.

IN PLAIN ENGLISH

This is the first study to actually prove (via causal tests, not just correlation) whether transformers truly generalize arithmetic across number systems. Training all four bases with equal amounts of data rules out the boring explanation that any difference is just due to one base having more training examples than another. The four chosen bases form a spectrum from 'very different-looking' (binary) to 'fairly similar-looking' (octal, hex) compared to decimal, which lets the researchers check if sharing tracks how similar the symbols look.

ORIGINAL TEXT

If shared circuits emerge despite large differences in token vocabulary and carry structure (as between binary and decimal), this constitutes strong evidence that transformers can learn abstract positional arithmetic. If circuits are surface-specific, this places a fundamental limit on the generality of the algorithms identified in prior mechanistic interpretability work. The methodology provides a reusable framework for studying surface-form invariance in transformer circuits beyond arithmetic.

IN PLAIN ENGLISH

Two possible outcomes, both interesting: (1) if the SAME internal machinery handles binary and decimal — which barely share any symbols — that's strong proof the model really learned abstract math. (2) If each base gets its own separate machinery, that means previous claims of 'transformers learned arithmetic' were overstated — they only learned a decimal-specific trick. Either way, the testing method itself can be reused to study other tasks beyond just arithmetic (e.g., does a model treat synonyms or different languages the same way internally?).

3. Related Work

Mechanistic interpretability of arithmetic

ORIGINAL TEXT

Nanda et al. [2023] reverse-engineered the circuit for modular arithmetic in small transformers, identifying a Fourier-based algorithm whose emergence coincides with a grokking phase transition. Kantamneni and Tegmark [2025] extended this to integer addition in larger models, showing that numbers are represented as a generalized helix and addition is implemented via a Clock algorithm verified with causal interventions. Quirke and Barez [2024] analyzed addition circuits in GPT-2, identifying carry-related structure in MLP layers. All of this work studies a single numeral system (decimal), leaving open whether the identified algorithms are base-specific or abstract.

IN PLAIN ENGLISH

Three earlier studies, summarized: (1) Nanda's team found that a small model doing modular math uses wave-pattern (Fourier) math internally, appearing right when grokking happens. (2) Kantamneni & Tegmark found that bigger models represent numbers as points arranged on a spiral/helix shape, and add them using something they call a 'Clock algorithm,' and they proved this causally (not just by observation). (3) Quirke & Barez found carry-related structure specifically inside the MLP (feedforward) layers of GPT-2. The catch: all three only ever looked at decimal, so none of them can say whether what they found is decimal-specific or universal.

Grokking and algorithmic generalization

ORIGINAL TEXT

Power et al. [2022] first characterized grokking — the delayed generalization phenomenon in which a model first memorizes training data and only later transitions to an algorithm that generalizes. Nanda et al. [2023] showed that grokked models develop interpretable circuits, making them strongly preferred for mechanistic analysis. Weight decay is the primary driver of grokking [Power et al., 2022]: it penalizes large weights and pushes the model toward compact, weight-efficient solutions.

IN PLAIN ENGLISH

Power et al. were the ones who first named and described 'grokking': a model memorizes first, then much later suddenly switches over to a real generalizable rule. Nanda's team then showed that once a model groks, its internals become much easier for humans to interpret — which is exactly why this whole field likes studying grokked models. And the main reason grokking happens at all is 'weight decay,' a training technique that discourages the model from using unnecessarily large numbers internally, nudging it toward simpler, more elegant solutions.

Shared circuits across tasks

ORIGINAL TEXT

Recent work has shown that models trained on related tasks can develop shared circuits. Quirke and Barez [2024] found shared carry and borrow circuits in models trained on both addition and subtraction. [Another study] found that subject-verb agreement circuits in English and Spanish are nearly identical and causally transferable. No prior work has studied circuit sharing across surface representations of numbers.

IN PLAIN ENGLISH

There's precedent for models sharing internal machinery across related but different tasks: Quirke & Barez found that the carrying mechanism (addition) and the borrowing mechanism (subtraction) actually overlap inside the same model. Separately, another study found that grammar-checking circuits for matching subjects and verbs are nearly identical between English and Spanish, and you can even swap them between the two languages and it still works. But nobody, until this paper, has checked whether this kind of sharing also happens across different ways of writing numbers.

ORIGINAL TEXT

Gap addressed by this work. No prior work has applied causal circuit analysis to arithmetic across multiple numeral bases, and no prior work has asked whether the algorithms identified in the grokking literature are surface-specific or genuinely abstract. This is the gap we address.

IN PLAIN ENGLISH

In one sentence: nobody has run causal, 'open-the-hood' tests on a model that does arithmetic in more than one number system — and that's precisely the gap this paper is built to fill.

4. Research Questions

ORIGINAL TEXT

RQ1: Does a single small transformer reliably grok addition across all four bases simultaneously when trained on a balanced corpus?

RQ2: Which attention heads and MLP layers contribute most to the correct-answer logit for each base, as measured by direct logit attribution?

RQ3: Are the components identified in RQ2 consistent across bases — i.e., does the model use the same circuit for addition regardless of the numeral base?

RQ4: At which layer does the model's intermediate representation of operand values converge to a shared encoding, if at all?

RQ5: Can activations from one base's forward pass causally substitute for another base's activations at shared components, without degrading accuracy?

IN PLAIN ENGLISH

Five questions, building on each other like a staircase:

- RQ1 (Can it learn?): Will one single model actually succeed at learning all four bases properly, not just memorize them?
- RQ2 (What's doing the work?): For each base separately, which specific parts of the network (attention heads, MLP layers) are most responsible for getting the right answer?
- RQ3 (Is it the same parts?): Are those important parts identified in RQ2 the SAME parts across all four bases, or different per base?
- RQ4 (Where does it become abstract?): At which specific layer, if any, does the model stop thinking in base-specific terms and start using one shared internal number representation?
- RQ5 (Can you prove it?): The ultimate test — if you literally swap the internal signal from a binary-addition pass into a decimal-addition pass at the shared components, does the decimal answer still come out correct? This is the strongest possible proof of genuine sharing.

5. Experimental Setup

Numeral Bases

ORIGINAL TEXT

We study four positional numeral bases chosen for their gradient of token overlap: Binary (digits 0,1; minimal overlap with decimal), Octal (digits 0–7; high overlap), Decimal (digits 0–9; the anchor base), and Hexadecimal (digits 0–9, A–F; partial overlap — 10 shared digits plus 6 new letter symbols). For example, $13 + 11 = 24$ in decimal becomes $1101 + 1011 = 11000$ in binary, $15 + 13 = 30$ in octal, and $D + B = 18$ in hexadecimal.

IN PLAIN ENGLISH

Four bases, picked specifically to form a spectrum of 'how much do their symbols overlap with decimal': binary overlaps the least (only shares 0 and 1), octal overlaps a lot (shares 0–7), decimal is the reference point, and hexadecimal overlaps partially (keeps all ten decimal digits but adds six brand-new letter symbols A through F). The example shows the exact same real-world sum (thirteen plus eleven) written out completely differently in each base.

ORIGINAL TEXT

This gradient is central to the experimental design. If circuit sharing correlates with token overlap, the model's internal algorithm is likely driven by shared surface tokens rather than abstract magnitude. If circuit sharing is constant across the gradient — strong between binary and decimal despite minimal token overlap — this is strong evidence for a genuinely base-agnostic algorithm.

IN PLAIN ENGLISH

This spectrum is the whole point of the design: if sharing goes up and down depending on how similar the symbols look (more sharing for octal/hex, less for binary), that means the model's 'trick' is really just about matching shared symbols, not true math understanding. But if sharing stays strong even for binary — the base that looks the LEAST like decimal — that's the strongest possible evidence the model has a genuine, abstract sense of numbers.

ORIGINAL TEXT

Octal provides an additional sharp test: $15 + 13 = 30$ in base 8, meaning the same underlying computation as $13 + 11 = 24$ in decimal produces entirely different tokens and a carry at a different position. This creates 'same computation, different surface' pairs that are useful for activation patching. Hexadecimal introduces six new digit tokens (A–F) not present in any other base, testing whether the circuit generalizes to novel symbols that have no analog in the model's other training data.

IN PLAIN ENGLISH

Octal gives a particularly clever trick case: thirteen plus eleven equals twenty-four no matter what base you write it in, but the octal version ($15 + 13 = 30$) looks nothing like the decimal version on the page, and the carry even happens in a different digit slot. This 'same math, totally different look' pair is gold for testing whether the model's internal representation is really about the abstract value or just the visible symbols. Hexadecimal adds a different kind of test: it throws in six entirely new symbols (A-F) that don't exist anywhere else in training, to see if the shared circuit can cope with totally novel symbols.

Prompt Format

ORIGINAL TEXT

Each prompt is formatted as $k = 8$ few-shot examples followed by a test problem: $x1 + y1 =$ (answer), ..., $x8 + y8 =$ (answer), $x + y =$ (blank for the model to fill in), where all operands and answers are expressed in the same base throughout the prompt. No explicit base-indicator token is included. The model must infer the base solely from the digit characters present in the few-shot examples — a model that succeeds has necessarily learned to condition its arithmetic on the surface representation of numbers.

IN PLAIN ENGLISH

Every test is set up as 'few-shot learning': show the model 8 fully solved addition examples in one base, then give it a 9th problem in the same base with the answer left blank for it to fill in. Importantly, the model is NEVER told outright which base it's looking at — it has to figure that out purely by noticing which digit symbols appear in the examples. So if the model gets the right answer, it proves it's actually paying attention to which symbols it's seeing and adjusting its math accordingly.

ORIGINAL TEXT

The few-shot examples for each prompt are drawn randomly from the training operand partition, subject to the constraint that at least two of the eight examples involve a carry, ensuring the in-context demonstrations adequately represent the carry structure of the base. The test problem operands are drawn from the held-out partition, preventing memorization of specific input-output pairs.

IN PLAIN ENGLISH

The 8 example problems shown to the model aren't totally random — at least 2 of them are guaranteed to require carrying a digit, so the model can't get away with only seeing 'easy' no-carry examples. Meanwhile, the actual test question (the one without a given answer) always comes from a separate pool of numbers the model never saw during training, so it can't just be remembering a fixed answer — it has to actually compute it.

ORIGINAL TEXT

For circuit analysis, the few-shot context introduces a confound: activations at the $=$ position are a function of both the test problem and the eight preceding examples. To isolate the contribution of the test problem, we fix the eight few-shot examples to a canonical set per base during evaluation, varying only the test operands. This ensures that differences in activations across prompts reflect the test problem alone, making the LASSO regression and ablation results interpretable as properties of the test computation rather than of the context.

IN PLAIN ENGLISH

Here's a subtle technical problem they had to solve: when you look at the model's internal signals right at the '=' sign, those signals are influenced by both the new test problem AND the 8 example problems shown before it. If the 8 examples kept changing randomly across tests, you couldn't tell whether a difference in internal signal was caused by the new problem or by which examples happened to come before it. Their fix: lock the same fixed set of 8 examples for every test within a given base, and only vary the actual test question. That way, any change you observe in the internals is guaranteed to be about the test problem itself, not noise from random examples.

Training Data

ORIGINAL TEXT

We construct a synthetic corpus of addition problems across all four bases. Operands are integers drawn uniformly from [0, 99] in decimal — equivalently, from [0, 1100011] in binary, [0, 143] in octal, and [0, 63] in hexadecimal. We fix the integer range rather than the digit count per base: the resulting variation in sequence length across bases (operands span at most 7 binary digits, 3 octal digits, 2 decimal digits, or 2 hexadecimal digits) is intentional. It reflects the genuine difference in surface representation between bases, and any shared circuit must be robust to it. Sums reach at most 198, requiring at most 8 binary digits, 3 octal digits, 3 decimal digits, or 2 hexadecimal digits.

IN PLAIN ENGLISH

They build their own training dataset of addition problems. The actual real-world values of the numbers used (0 to 99) are kept identical across all four bases — it's only how those same numbers are WRITTEN that changes. This is a deliberate choice: it means a number like 99 takes only 2 digits in decimal but up to 7 digits in binary, so the model has to deal with very different sequence lengths depending on the base — and any truly shared circuit has to be flexible enough to handle that.

ORIGINAL TEXT

Carry problems — those for which the carry propagates at least one position — are oversampled to ensure the model cannot avoid learning carry structure. We use a character-level tokenizer with one token per character in the vocabulary {0–9, A–F, +, =, comma, space} (21 tokens total). Prompts are distributed equally across all four bases at the problem level. All bases see identical operand distributions. A held-out operand split per base prevents memorization of specific input-output pairs.

IN PLAIN ENGLISH

Problems that require carrying (like $7 + 5 = 12$, where the 1 carries over) are shown to the model more often than they'd naturally occur, specifically so it can't cheat by ignoring carrying altogether. The model reads input one character/digit at a time (called 'character-level tokenization'), using a small fixed alphabet of 21 possible symbols. All four bases get exactly equal amounts of training data, and a chunk of number-pairs is reserved purely for testing so the model truly has to compute answers rather than recall memorized ones.

Model Selection

ORIGINAL TEXT

We conduct a grid search over hidden dimension $d \in \{32, 64, 128, 256\}$, number of layers $L \in \{1, 2, 3\}$, and number of attention heads $H \in \{1, 2, 4, 8\}$, with MLP intermediate dimension fixed at $4d$. All models are trained with AdamW under a cosine learning rate schedule with high weight decay to reliably induce grokking.

IN PLAIN ENGLISH

Rather than picking one model size by guesswork, they systematically try out many combinations of size settings (how wide the model is, how many layers, how many attention heads) and train each combination using standard, modern training settings, with the weight decay turned up high specifically to make grokking reliably happen.

ORIGINAL TEXT

We require grokking to occur for each base independently: validation accuracy must saturate strictly after training accuracy for each of the four bases. We select the smallest model by parameter count satisfying this criterion across all four bases simultaneously, with at least 95% exact-match accuracy on the held-out set per base. The minimum-over-bases criterion ensures genuine multi-base generalization rather than average performance.

IN PLAIN ENGLISH

They have a strict rule for what counts as a valid model: for EVERY single base individually (not just on average), the model must show the classic grokking pattern (memorize first, generalize later) and hit at least 95% accuracy on brand-new test problems. Out of all model sizes that pass this bar for all four bases, they pick the smallest one. Smaller models tend to use cleaner, easier-to-interpret internal logic, which is exactly what this kind of circuit-analysis study needs.

6. Methodology

ORIGINAL TEXT

Our analysis follows six steps. Step 0 defines the units of analysis and collects the activations that all subsequent steps operate on. Steps 1 and 2 identify and causally confirm the circuit: LASSO proposes a sparse candidate set, and ablation establishes which candidates are causally necessary. Steps 3–5 characterize the confirmed circuit: the logit lens traces how representations evolve across layers, DLA attributes signed contributions to individual components, and activation patching tests whether shared components encode genuinely base-agnostic information.

IN PLAIN ENGLISH

The overall plan is a 6-step pipeline. Step 0 is just data collection. Steps 1-2 are about FINDING the shared circuit (first a rough statistical guess, then a real causal test to confirm it). Steps 3-5 are about UNDERSTANDING that confirmed circuit in more depth — tracing how it evolves layer by layer, measuring exactly how much each piece contributes, and finally directly testing whether it encodes truly abstract, swappable information.

Step 0: Activation Extraction

ORIGINAL TEXT

We use the term neuron to refer to either an individual scalar pre-activation in the MLP intermediate layer (4d per layer) or a full attention head output vector treated as a single unit (H per layer). Treating the attention head as an atomic unit is consistent with prior circuit-level analyses, where heads are the natural unit of attention computation. Together, each layer exposes $4d + H$ neurons under this definition.

IN PLAIN ENGLISH

They define 'neuron' loosely to mean two different kinds of building blocks: individual numbers inside the MLP (feedforward) layer, or an entire attention head treated as one single unit (rather than breaking it down further). This matches how other researchers usually study attention — as whole heads, not sub-pieces.

ORIGINAL TEXT

Each prompt consists of eight few-shot examples followed by the test problem. We extract activations at five semantically meaningful token positions within the test problem: the final character of operand a, the final character of operand b, the operator token (+), the = token (the last input token before generation begins), and the first generated answer token. Steps 1–5 focus on the = position, which aggregates the full problem context — including the few-shot demonstrations — immediately prior to generation.

IN PLAIN ENGLISH

Instead of recording every single internal signal everywhere, they zoom in on five specific, meaningful moments in each problem: the end of the first number, the end of the second number, the plus sign, the equals sign, and the very first digit of the answer the model produces. Most of the deeper analysis (Steps 1-5) specifically focuses on the moment right at the equals sign, since that's the point where the model has seen the whole problem and is about to start answering — effectively its 'final summary' of the question.

ORIGINAL TEXT

At each position and each layer we extract three activation types: the Residual stream (the accumulated representation at the output of each transformer block), the MLP intermediate activations (the scalar pre-activations entering the down-projection of the MLP sublayer), and the Attention head outputs (the output of each attention head prior to the output projection). We collect activations over 10,000 evaluation prompts per base, yielding 40,000 prompts in total.

IN PLAIN ENGLISH

For each of those five key moments, at every layer of the model, they record three different 'flavors' of internal signal: the running cumulative representation (residual stream), the raw numbers inside the feedforward layer, and the raw output of each attention head. They do this across 10,000 test problems per base (40,000 total), building up a big dataset of internal signals to analyze in the following steps.

Step 1: Identify Candidate Neurons via LASSO

ORIGINAL TEXT

We fit an ℓ_1 -regularized linear regression for each base, predicting the correct answer token's logit from the neuron activations at the $=$ position. The feature vector comprises, for each layer, the 4d MLP intermediate scalars and the ℓ_2 -norm of each attention head output, concatenated across all layers into one large vector. We solve a standard LASSO optimization, with the regularization strength chosen via 5-fold cross-validation. The support (non-zero entries) of the solution constitutes the candidate circuit for that base. Neurons appearing in the candidate set for all four bases are promoted to cross-base candidates and carried forward to Step 2.

IN PLAIN ENGLISH

LASSO is a statistical technique that's good at automatically picking out 'which ingredients actually matter' out of a huge pile of possible variables, while zeroing out the unimportant ones. Here, they feed it all the recorded internal signals (from every neuron, every layer) and ask it to predict how strongly the model is leaning toward the correct answer. Whichever neurons LASSO decides matter become the 'candidate list' for that base. Then they check: which neurons show up on the candidate list for ALL FOUR bases? Those become the prime suspects for being part of a shared circuit, which then get tested more rigorously in Step 2.

Step 2: Confirm Causal Necessity via Ablation

ORIGINAL TEXT

LASSO identifies neurons that are linearly predictive of the correct answer but not necessarily causally necessary. For each neuron in the cross-base candidate set, we zero out its activation at the $=$ position during the forward pass and measure the resulting drop in exact-match accuracy, independently per base. Neurons are classified as: Universally shared (ablation degrades accuracy in all four bases), Base-specific (ablation degrades accuracy in only one or two bases), or Non-causal (ablation has no significant effect). The universally shared neurons that survive this filter constitute the causally confirmed circuit, characterized further in Steps 3–5.

IN PLAIN ENGLISH

Here's the catch with LASSO: it can flag a neuron as 'important' just because its value happens to correlate with the right answer, even if it's not actually CAUSING that answer (correlation isn't causation). So in this step, they actually intervene: for each candidate neuron, switch it off (set it to zero) and see if the model's accuracy actually drops. If turning it off hurts accuracy in ALL four bases, it's truly 'universally shared.' If it only hurts one or two bases, it's 'base-specific.' If turning it off changes nothing, it was a false positive from LASSO ('non-causal'). Only the survivors of this real test — the universally shared ones — become the officially 'confirmed circuit' that gets deeper analysis next.

Step 3: Trace Layer-wise Evolution via Logit Lens

ORIGINAL TEXT

We apply the logit lens to the confirmed circuit components, projecting the residual stream at each layer onto the output vocabulary and decoding the top token. For each base, we track: (a) whether the decodable token corresponds to the correct answer, and (b) whether it is expressed in the input base or a different one. If intermediate representations decode as the correct digit in a different base than the input — e.g., a binary-input problem whose middle-layer representation decodes as a decimal numeral — this constitutes evidence of an internal base-neutral encoding, analogous to the English-pivot behavior observed in multilingual models. This step localizes the layer at which base-specific surface representations give way to a shared numerical encoding within the confirmed circuit.

IN PLAIN ENGLISH

The 'logit lens' is a way of peeking at what the model would guess if you forced it to answer right now, using its half-finished, in-progress signal at any given layer (rather than waiting for the very final layer). They apply this layer by layer and check two things: is the guess correct, and what BASE is the guess expressed in? If a problem given in binary shows the model's middle-layer 'guess' actually looking like a DECIMAL number (even though the input and the correct final output are binary), that's a hint the model internally converts everything to one shared, neutral numerical format partway through — similar to how some multilingual AI models seem to internally 'think in English' before translating their final answer into another language. This step pinpoints exactly which layer this base-neutral switch happens, if it happens at all.

Step 4: Attribute Contributions via Direct Logit Attribution

ORIGINAL TEXT

We apply direct logit attribution (DLA) to the confirmed circuit to decompose each component's signed contribution to the correct-answer logit. Because the residual stream is a sum of component outputs, the logit for any token can be written as an exact linear sum: for each attention head and MLP sublayer in the confirmed circuit, we project its output through the unembedding matrix to obtain a signed scalar contribution. We compute DLA per (base, problem) across all four bases, identifying which components contribute most uniformly to the correct-answer logit regardless of base.

IN PLAIN ENGLISH

Because of how transformers are mathematically built, the model's final 'score' for any candidate answer can be split apart exactly into how much each individual component (each attention head, each MLP layer) added or subtracted toward that score — with no approximation needed. DLA does exactly this split. By running it across all four bases, they get a precise scoreboard of which components push hardest and most consistently toward the right answer, no matter which base is being used.

Step 5: Test Representational Sharing via Activation Patching

ORIGINAL TEXT

Ablation establishes which neurons are shared; activation patching establishes what they encode. We construct matched problem pairs: two problems representing the same underlying addition (same integer operands and answer) expressed in two different bases. For each component in the confirmed circuit, we replace its activation from the base-A forward pass into the base-B forward pass and measure whether the correct base-B answer is still produced. We run this for all six base-pair combinations and at each layer to localize where base-agnostic representations emerge.

IN PLAIN ENGLISH

Knowing a neuron is 'necessary' (from Step 2) doesn't tell you WHAT information it's actually holding. So here, they do the most aggressive test possible: take two problems that represent the exact same real-world sum but written in two different bases, then literally swap the internal signal from one base's forward pass into the other base's forward pass at a shared component. If the model still produces the correct answer (in the second base's format) after this swap, that's almost airtight proof the component holds truly abstract, interchangeable information about the math — not something tied to one specific base's symbols. They repeat this for every pair of the four bases and at every layer, to map out exactly where this 'universal translator' behavior kicks in.

ORIGINAL TEXT

The octal test case provides a particularly sharp version of this experiment: since $15 + 13 = 30$ in base 8 but $13 + 11 = 24$ in base 10, patching activations from the decimal forward pass into the octal forward pass tests whether shared components encode the correct base-conditioned answer or merely the underlying integer sum. The distinction reveals whether the representation is truly abstract or magnitude-encoding only.

IN PLAIN ENGLISH

Octal is the perfect stress test here because the SAME real numbers (13 and 11) produce completely different-looking equations in octal versus decimal. If you swap a decimal signal into an octal forward pass and the model still spits out the correct OCTAL-formatted answer (30, not 24), that proves the shared component understands both the raw quantity AND knows how to express it correctly in whatever base it's currently working in — truly abstract reasoning, not just a magnitude counter that doesn't know how to format its output.

7. Expected Outcomes

ORIGINAL TEXT

The six-step pipeline is informative regardless of outcome. We anticipate one of three primary findings. Shared circuit: the same small set of attention heads and MLP neurons drives addition in all four bases. DLA scores are consistent across bases, ablation produces uniform accuracy drops, and activation patching succeeds across base pairs. This would mean the model learned a genuinely base-agnostic addition algorithm — the carry structure is shared even when the surface tokens differ.

IN PLAIN ENGLISH

No matter what they find, the study still teaches us something useful. The first possible outcome is the cleanest: a fully SHARED circuit, where the exact same handful of components do all the work in every base, all the tests (DLA, ablation, patching) line up consistently, and patching always succeeds. This would mean the model truly learned one abstract addition algorithm that works regardless of which symbols it's reading or writing.

ORIGINAL TEXT

Partially shared circuit. Middle-layer components are shared (magnitude and carry encoding) while early and late components are base-specific (digit embedding and answer decoding). This would localize precisely where the abstraction over surface form occurs, analogous to the language-peripheral, content-central structure observed in multilingual models.

IN PLAIN ENGLISH

The second possible outcome is more nuanced: the MIDDLE of the model (where it figures out magnitude and handles carrying) is shared across bases, but the very beginning (reading the raw digit symbols) and the very end (writing out the answer in the right symbols) stay base-specific. This would tell us exactly where the 'translation boundary' sits — echoing how some multilingual models seem to handle the input/output language at the edges but reason about meaning in a shared, language-neutral way in the middle.

ORIGINAL TEXT

Base-specific circuits. Separate subnetworks develop for each base, with different components responsible for each. Activation patching would fail. This would mean the algorithms identified in prior single-base grokking studies are surface-specific, placing a fundamental limit on their generality.

IN PLAIN ENGLISH

The third possible outcome is the least flattering for the 'transformers really understand math' story: completely SEPARATE circuits per base, with patching simply failing across the board. This would mean every prior single-base grokking study (which only ever tested decimal) was likely discovering a decimal-specific trick rather than a universal algorithm — a real limitation on how far those earlier results can be trusted to generalize.

ORIGINAL TEXT

The gradient structure of the four bases provides a secondary test: if circuit sharing correlates with token overlap (binary least shared, octal most shared relative to decimal), this suggests the sharing is driven by co-occurrence of surface tokens rather than abstract numerical structure.

IN PLAIN ENGLISH

There's also a bonus diagnostic baked into the design: if the AMOUNT of sharing they measure tracks how visually similar a base's symbols are to decimal's (binary sharing the least, octal sharing the most), that's a red flag that the 'sharing' is really just a side-effect of overlapping symbols, rather than true abstract number sense.

8. Further Experiments

ORIGINAL TEXT

The primary study establishes which components form the shared circuit and whether it generalizes across bases. The following extensions pursue two deeper goals: understanding the mechanism the shared circuit implements, and testing the boundaries of that mechanism under more demanding conditions.

IN PLAIN ENGLISH

Once the main study figures out WHICH components are shared, these follow-up ideas push further in two directions: figuring out exactly HOW those shared components do their math internally, and stress-testing the limits of that sharing under harder conditions.

Recovering the underlying algorithm

ORIGINAL TEXT

Identifying the confirmed circuit tells us which components are necessary but not what algorithm they implement. Prior work on single-base arithmetic has characterized specific algorithmic structures: Nanda et al. [2023] found that grokked transformers implement modular addition via a Fourier basis in weight space, and Kantamneni and Tegmark [2025] found that integer addition is implemented via a Clock algorithm in which numbers are represented as points on a generalized helix. The central question this extension asks is: does the same algorithm appear across all four bases within the confirmed circuit, or do bases that share a circuit nonetheless implement it via structurally different internal representations?

IN PLAIN ENGLISH

Knowing WHICH neurons matter is different from knowing WHAT MATH TRICK those neurons are actually running. Earlier research found two specific tricks in decimal-only models: a wave-pattern (Fourier) trick, and a spiral/clock-shaped geometric trick for representing numbers. This follow-up asks: even if all four bases share the same components, do those components run the exact same internal trick in every base, or could they share the 'hardware' while running subtly different internal 'software' per base? That second possibility would be a more surprising and nuanced finding.

ORIGINAL TEXT

Concretely, we will analyze the weight matrices of components in the confirmed circuit for Fourier structure, visualize the geometry of operand representations (testing for helical structure), and apply Discrete Fourier Transforms to MLP neuron activations as a function of operand value. If the same frequencies dominate across all four bases, the model has not only shared the circuit but implemented a single base-agnostic algorithm within it. If different frequencies or geometric structures appear per base despite shared components, the circuit sharing is structural but the algorithm is base-specific.

IN PLAIN ENGLISH

Practically, this means running mathematical pattern-detection tools (like checking for wave frequencies or spiral/helix shapes) on the shared circuit's internal weights, across all four bases. If they find the exact same wave frequencies or shapes everywhere, that's the deepest possible proof of true universal arithmetic understanding. If the shapes differ per base even though the same neurons are 'doing the work,' that's a subtler finding: the model reuses the same wiring, but runs a slightly different program on that wiring depending on the base.

Subtraction

ORIGINAL TEXT

Subtraction is the most natural extension because borrowing is structurally carry run in reverse. The cross-base question applies directly: does the model develop a shared borrow circuit across all four bases? The more interesting secondary question is whether the carry and borrow circuits are unified — the same components distinguished by a sign or direction signal — or separate but each base-agnostic. A unified circuit would constitute stronger evidence for abstract positional-arithmetic computation. Mechanistic analysis of the borrow circuit via the same six-step pipeline would further test whether the Fourier or helix structure found for addition also underlies subtraction, or whether a qualitatively different algorithm emerges.

IN PLAIN ENGLISH

Subtraction is a natural next step because 'borrowing' (like in $12 - 7$) is basically carrying run backwards. The same cross-base question applies: is there a shared borrowing circuit across all four bases too? Even more interesting: is the SAME machinery used for both carrying (addition) and borrowing (subtraction), just with some kind of 'direction flag' flipped, or are they two completely separate-but-each-still-shared circuits? If carrying and borrowing turn out to be literally the same machinery, that's even stronger evidence the model has a genuinely abstract sense of positional arithmetic, not just two unrelated memorized tricks.

Roman numerals as a non-positional boundary

ORIGINAL TEXT

Roman numerals (e.g., $\text{III} + \text{V} = \text{VIII}$) represent the sharpest possible test of the mechanism's limits: they are non-positional, so the carry-based algorithm identified in the primary study cannot apply. If the LASSO and ablation pipeline identifies any overlap between the Roman numeral circuit and the confirmed circuit, this would indicate a more abstract magnitude-level representation than carry structure alone. If circuits diverge entirely, Roman numerals establish a clean lower bound on what surface properties a shared mechanism requires.

IN PLAIN ENGLISH

Roman numerals ($\text{III} + \text{V} = \text{VIII}$) are a brutal stress test because they don't work like normal place-value number systems at all — there's no carrying, no digit positions that mean '10s' or '100s.' So the carry-based trick the model presumably learned simply cannot apply here. If, surprisingly, there's STILL some overlap between the Roman-numeral circuit and the main shared circuit, that would suggest the model has an even deeper sense of 'quantity' that goes beyond just handling carries. If there's zero overlap, that tells us exactly where the limits of the shared mechanism are — useful information either way.

Operand range scaling

ORIGINAL TEXT

The primary study uses operands from $[0, 99]$ in decimal, giving sums of at most 198. Scaling to $[0, 999]$ adds one decimal digit but roughly three binary digits, creating increasingly unequal sequence lengths across bases. Whether the confirmed circuit survives this asymmetry — and whether the Fourier or helix structure remains intact at larger operand magnitudes — tests how deeply the model has abstracted over both surface form and problem scale.

IN PLAIN ENGLISH

The main study only uses numbers up to 99. This follow-up pushes that up to 999, which only adds one extra digit in decimal but roughly three extra digits in binary — so the mismatch in sequence length between bases gets much more extreme. This tests whether the shared circuit (and whatever wave/spiral pattern it uses) can survive that increased imbalance, or whether it was secretly relying on the bases being roughly similar in length all along.